

A
Synopsis on
INTENT2CAPABILITY

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering

by

Ashutosh Shukla [2300970100069]
Ayush Narayan Gupta [2300970100074]
Ayush Kumar [2300970100073]
Ankit Mishra [2300970100047]

Semester – VII

Under the Supervision of
Dr. Amrita Jyoti



Galgotias College of Engineering & Technology
Greater Noida 201306

Affiliated to



Dr. APJ Abdul Kalam Technical University, Lucknow

ABSTRACT

The rapid proliferation of generative artificial intelligence (AI) and autonomous coding assistants has fundamentally transformed contemporary software engineering by automating the synthesis of complex multi-file codebases directly from informal natural-language prompts. However, this development paradigm introduces a critical, unaddressed security challenge: AI-generated software frequently acquires execution authorities, file system privileges, and network capabilities that significantly exceed and diverge from the developer's declared requirements.

Conventional Static and Dynamic Application Security Testing (SAST/DAST) tools are designed to identify canonical syntactic vulnerability patterns, memory corruption flaws, and known Common Vulnerabilities and Exposures (CVEs). However, these systems remain inherently agnostic to whether an exercised runtime capability is legitimate relative to stated application intent. Consequently, an AI-synthesized program may execute arbitrary background subprocesses, exfiltrate host environment secrets, or open unauthorized outbound network sockets without triggering traditional vulnerability scanners, provided that the underlying syntax is standard and well-formed.

This project presents **Intent2Capability**, an automated capability-divergence analyzer and authority assurance framework designed to enforce the principle of least privilege in AI-generated software. The proposed framework implements a multi-stage verification pipeline: (1) an LLM-based Intent Decomposer with constrained Pydantic schema validation that extracts formal Expected Capability Specifications (C_{exp}) from natural-language requirements; (2) a multi-language static analyzer utilizing Tree-sitter Concrete Syntax Tree (CST) queries to extract potential Static Capabilities (C_{stat}); (3) a containerized, unprivileged Docker execution harness equipped with Linux system-call tracing (`strace`) and intercepting network proxy monitoring to capture Dynamic Capabilities (C_{dyn}); and (4) a deterministic Graph Alignment Engine that computes the capability divergence set ($\Delta = (C_{stat} \cup C_{dyn}) \setminus C_{exp}$).

Divergences are categorized into structured risk tiers (Justified, Conditionally Justified, Suspicious, Unjustified, and Ambiguous) accompanied by line-level source code and system-call evidence traces. The framework integrates into continuous integration (CI/CD) pipelines as an automated pull-request blocker. Intent2Capability will be evaluated across a controlled benchmark suite of 50 multi-domain applications spanning JavaScript/TypeScript and Python codebases synthesized by state-of-the-art LLMs. The project establishes an empirical, defensible assurance mechanism that prevents ambient privilege inflation without asserting universal program verification.

Keywords: *AI-Generated Software Security, Intent-to-Capability Alignment, Principle of Least Privilege, Tree-sitter AST Parsing, Sandboxed System-Call Tracing, Capability Divergence, Authority Assurance.*

1. INTRODUCTION

The integration of Large Language Models (LLMs) and autonomous coding assistants into modern software development workflows has reached widespread adoption. Tools such as GitHub Copilot, Claude Engineer, Devin, and Cursor allow developers to construct multi-file applications, generate complex API wrappers, and automate system scaffolding directly from informal natural-language prompts. While these generative systems drastically reduce time-to-market and boilerplate implementation overhead, they shift the developer's primary role from manual implementation to high-level architectural specification.

1.1 Security Implications of AI-Synthesized Code and Authority Inflation

In traditional software development, human engineers explicitly select third-party libraries, configure operating system access boundaries, and define network endpoints. In contrast, neural code synthesis models operate by predicting statistically probable token sequences learned over vast, heterogeneous open-source corpora. Consequently, generated code frequently incorporates unrequested generic boilerplate logic, broad operating system permissions, unprompted telemetry endpoints, or aggressive background shell commands to ensure immediate functional execution.

When developers review AI-generated code, verification is predominantly confined to functional correctness—such as verifying graphical interface rendering or ensuring that unit tests pass. Ambient execution authority, underlying dependency graphs, and background system-call behaviors are rarely scrutinized. This introduces a major security challenge known as *authority inflation*, wherein the synthesized software acquires ambient privileges that far exceed its functional necessity.

1.2 Intent-Relative Capability Analysis vs. Vulnerability Scanning

A fundamental principle in application security is that **dangerousness is not identical to unjustifiedness**. Executing a system operation—such as writing to local disk, spawning a subprocess, reading an environment variable, or opening an outbound network socket—is not intrinsically malicious. In an administrative backup utility, disk I/O and process execution are expected and justified. However, in an offline scientific calculator, markdown parser, or unit converter, the identical set of operations constitutes an unjustified capability divergence and an immediate security vulnerability.

Traditional Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools are structurally incapable of resolving this distinction because they operate under global, context-free rules. They lack any model of the developer's original natural-language intent. Intent2Capability addresses this semantic gap by operationalizing requirement-relative capability analysis.

1.3 Formal Eight-Class Capability Taxonomy

To systematically represent and correlate execution authority across multi-language source code and low-level operating system events, Intent2Capability defines an eight-class capability taxonomy:

Table 1: Formal Eight-Class Capability Taxonomy and Detection Targets

Taxonomy Class	Normalized Actions	Detection Targets (AST & Syscalls)
1. Filesystem	READ_WORKSPACE, WRITE_WORKSPACE, READ_SYSTEM_FILE, DELETE_FILE	fs.readFile, open(), sys_enter_openat, sys_enter_unlink
2. Network	HTTP_GET, HTTP_POST, RAW_SOCKET, DNS_QUERY, EXTERNAL_EGRESS	fetch(), axios, requests.get, sys_enter_connect
3. Process Exec	SPAWN_SHELL, EXEC_BINARY, FORK_PROCESS, INVOKE_INTERPRETER	child_process.exec, subprocess.Popen, sys_enter_execve
4. Secret/ Creds	READ_ENV_VAR, READ_SSH_KEY, READ_DOTENV, ACCESS_VAULT	process.env, os.environ, access to ~/.ssh or .env
5. Database	CONNECT_DB, READ_TABLE, WRITE_TABLE, SCHEMA_ALTER	pg, mysql2, sqlite3, SQLAlchemy, ORM drivers
6. Package/ Runtime	DYNAMIC_IMPORT, NPM_INSTALL, PIP_INSTALL, EVAL_CODE	eval(), Function(), importlib, dynamic require()
7. Cloud/3rd- Party	S3_UPLOAD, AWS_IAM_ACCESS, STRIPE_CHARGE, SEND_EMAIL	@aws-sdk/client-s3, boto3, stripe, sendgrid
8. System & Host	READ_HOST_METADATA, KERNEL_INFO, DEVICE_ACCESS	os.platform(), os.networkInterfaces(), psutil

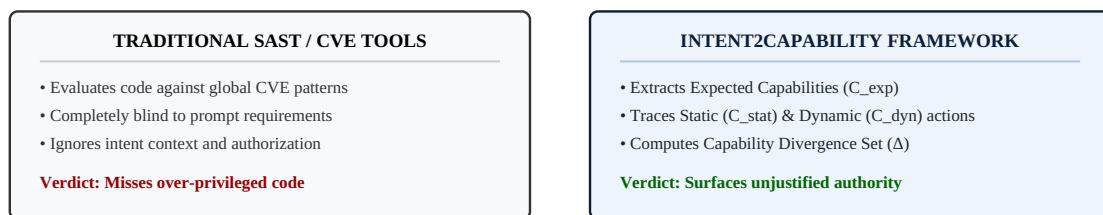


Figure 1: Paradigm Comparison between Traditional SAST and Intent2Capability Framework

1.4 Scope and Boundaries

Intent2Capability does not claim to replace established vulnerability scanners or provide formal mathematical proofs of total program correctness. By Rice's Theorem, non-trivial semantic properties of arbitrary programs cannot be decided statically with complete soundness and completeness. Instead, this project provides an empirical, lightweight alignment framework that surfaces capability inflation, gathers deterministic evidence, and enforces least-privilege guardrails during CI/CD execution.

2. LITERATURE SURVEY

2.1 Principle of Least Privilege and Classical Access Control

The foundational concept of execution authority restriction originates from the principle of least privilege formulated by Saltzer and Schroeder [1]. Their seminal work established that every program module, process, and user in a computing system must operate using the minimal set of privileges strictly necessary to complete its legitimate objective. In classical security engineering, least privilege was historically enforced through static Discretionary Access Control (DAC) or Mandatory Access Control (MAC) models, requiring system administrators to manually configure fine-grained permissions, capabilities, and access control lists prior to execution.

While effective in deterministic environments, manual access control models break down in rapid, automated software generation workflows. Modern generative AI coding agents synthesize code with dynamic execution flows and multi-layered dependency trees, rendering manual policy configuration impractical for continuous delivery pipelines.

2.2 Operating System Sandboxing and System-Call Interposition

To mitigate unauthorized program behaviors at runtime, operating systems research has extensively explored kernel-level monitoring and system-call interposition. Provos [2] pioneered policy-driven system-call filtering with *Systrace*, demonstrating that intercepting and validating system calls at the kernel boundary effectively prevents untrusted applications from executing unauthorized operations. Modern Linux isolation mechanisms—including seccomp-BPF filters, namespaces, and cgroups—provide robust primitives for constraining container execution.

However, existing sandbox architectures rely on fixed global policies (e.g., denying all root operations or restricting socket creation). They do not evaluate whether a particular system call is legitimate relative to the developer's original functional requirements. If a sandboxed utility is granted network access because some modules require it, the sandbox cannot distinguish between a legitimate weather API fetch and an unauthorized credential exfiltration payload.

2.3 Static Program Analysis and Code Property Graphs

Static program analysis provides compile-time techniques to analyze source code without execution. Classical static techniques analyze Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and Data Flow Graphs (DFGs) to detect syntactic errors, uninitialized variables, and unsafe memory operations.

Yamaguchi et al. [3] made a major structural contribution by introducing Code Property Graphs (CPGs), which unify ASTs, CFGs, and Program Dependence Graphs into a single joint data structure. This representation enables graph traversal queries to model complex vulnerability patterns and taint tracking across large codebases. Tools such as GitHub CodeQL, Snyk, and Semgrep build upon these foundational principles to identify Common Weakness Enumerations (CWEs) and data-taint paths.

Despite their effectiveness in identifying explicit software flaws (such as SQL injection, cross-site scripting, and buffer overflows), static analyzers are inherently limited when evaluating execution legitimacy. An AST query can confirm whether a function invokes `child_process.exec` or accesses `process.env`, but it cannot determine whether that invocation is authorized by the developer's high-level requirements. Furthermore, static analysis struggles to resolve dynamically computed strings, reflection, and obfuscated payloads, often producing false alarms or missing latent runtime capabilities.

2.4 Empirical Security Analysis of AI-Generated Software

The emergence of Large Language Models (LLMs) trained on vast code repositories has introduced new vectors for software insecurity. Pearce et al. [4] performed the first large-scale empirical evaluation of GitHub Copilot's code contributions across high-risk cybersecurity scenarios. Their findings demonstrated that across 89 distinct scenarios, approximately 40% of generated programs contained severe security weaknesses, including improper input validation, hardcoded credentials, and unsafe memory handling.

Subsequent user studies conducted by Sandoval et al. [5] examined human-AI interaction in security-critical programming tasks. Their results revealed that developers writing code with AI assistants frequently introduce subtle security regressions while exhibiting significant overconfidence in the safety and correctness of the generated artifacts.

2.5 Supply-Chain Risks and Agentic Privilege Bloat

As AI coding assistants have evolved from single-line autocompletion to autonomous multi-file agentic systems, the scope of generated code has expanded to include package management, environment variable parsing, and network communication. Kabore et al. [6] conducted a systematic literature review analyzing the security landscape of AI-based source code generators. Their survey categorized emerging threat models, highlighting that AI generators frequently introduce phantom dependencies, unvetted third-party packages, and excessive ambient permissions that human engineers fail to inspect during code review.

Siddiq et al. [7] specifically investigated capability boundaries and privilege inflation in neural code synthesis. Their empirical study demonstrated that LLMs frequently bundle unnecessary administrative and network privileges into generated software, creating expansive attack surfaces. This authority bloat provides malicious actors with avenues for supply-chain compromise, prompt injection exploitation, and environment reconnaissance.

2.6 Intent Formalization and Specification Mining

Bridging the gap between informal natural-language requirements and formal program specifications represents a premier challenge in computer science. Chen et al. [8] explored specification mining and behavioral intent verification for neural code synthesis, demonstrating that extracting verifiable semantic contracts from informal prompts is essential for ensuring code safety.

While theoretical intent formalization focuses on interactive theorem proving and temporal logic, practical DevSecOps pipelines require lightweight, automated mechanisms that can operate at scale without requiring manual formal proof authoring. Intent2Capability provides an empirical operationalization of this requirement by modeling intent as a structured capability graph and aligning it with observed static and dynamic program execution traces.

2.7 Comparative Synthesis of Related Work

Table 2 summarizes the operational mechanisms, strengths, and limitations of existing security approaches relative to the Intent2Capability framework:

Table 2: Comparative Analysis of Security Tooling and Methodologies

Tool / Approach	Primary Mechanism & Strengths	Limitation Addressed by Intent2Capability
GitHub CodeQL / Snyk [3, 4]	Code Property Graphs, taint tracking, and rule-based CVE detection.	Detects syntactic bugs; cannot evaluate if an action is justified by natural-language prompt requirements.
Semgrep Guardian [6]	Lightweight AST pattern matching tailored for generated code.	Enforces static global rules without requirement-relative context.
OS Sandboxes / eBPF [2, 7]	Kernel-level system call filtering and network isolation.	Constrains runtime execution against global policies without intent-aware boundary checks.
Formal Intent Mining [8]	Temporal logic and formal mathematical specification extraction.	Theoretically rigorous but computationally heavy; lacks empirical CI/CD capability alignment tooling.
Intent2Capability (Proposed)	Hybrid static AST + dynamic sandboxed syscall alignment engine.	Extracts prompt-relative capabilities and detects unjustified execution authority with line-level evidence.

2.8 Summary of Research Gap

Identified Research Gap: Current application security testing methodologies are bifurcated: static analyzers detect syntactic vulnerabilities without requirement awareness [3, 4], while sandboxes enforce system-call policies without verifying if runtime behavior matches user intent [2, 7]. There is an absence of an integrated, automated assurance framework that formalizes informal natural-language prompts into capability specifications and evaluates whether an AI-synthesized program's static AST structure and dynamic runtime traces exceed stated requirements [8]. Intent2Capability directly addresses this critical research gap.

3. PROBLEM FORMULATION: NEED AND SIGNIFICANCE OF PROPOSED RESEARCH WORK

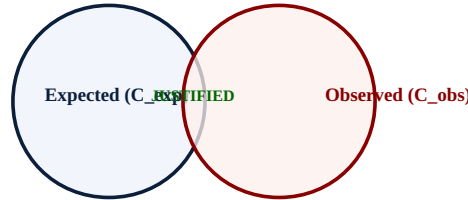
When software is synthesized automatically from informal natural-language prompts, a severe semantic disconnect arises between human conceptual intent and concrete execution authority. Existing application security methodologies fail to resolve this problem due to three fundamental limitations:

- **Limitation of Static Analysis Alone:** Static AST parsing identifies potential API calls across all control flow branches but cannot reliably resolve dynamically computed strings (e.g., `fetch(atob(...))`), runtime environment bindings, or dynamic URLs, leading to false positives or dead-code over-reporting.
- **Limitation of Dynamic Analysis Alone:** Dynamic sandboxing observes concrete runtime execution and exact system calls but is strictly constrained by test execution paths, failing to detect latent malicious or over-privileged logic present in unexercised branches.
- **Absence of Intent Awareness:** Existing security linters apply uniform rules; they cannot discern that an HTTP request is essential for a weather forecast module but constitutes a severe security divergence in an offline calculator.

3.1 Mathematical Formulation of Capability Divergence

Let R denote the natural-language requirement prompt provided by the developer, and let P denote the synthesized source code program. Let T represent the execution test suite or runtime harness. The formal verification model is defined as:

1. Expected Capabilities:	$C_{exp} = \Phi(R)$
2. Static Candidate Capabilities:	$C_{stat} = \Psi_s(P)$
3. Dynamic Observed Capabilities:	$C_{dyn} = \Psi_d(P, T)$
4. Total Observed Authority:	$C_{obs} = C_{stat} \cup C_{dyn}$
5. Capability Divergence Set:	$\Delta = C_{obs} \setminus C_{exp} = (C_{stat} \cup C_{dyn}) \setminus C_{exp}$



$$(C_{exp} \cap C_{obs}) \Delta = \text{Capability Divergence}$$

Figure 2: Capability Set Divergence Set-Theoretic Representation

Here, Φ represents the Intent Decomposition function mapping informal requirements into a constrained capability schema. Ψ_s represents the static AST extraction operator using Tree-sitter queries, and Ψ_d denotes dynamic observation via containerized kernel tracing. The capability divergence set Δ isolates all actions present in the codebase or exercised during execution that lack justification within C_{exp} . Any non-empty divergence set ($\Delta \neq \emptyset$) triggers an audit warning requiring human confirmation before deployment.

4. OBJECTIVES

To establish a comprehensive intent-to-capability assurance framework for AI-generated software, this project defines five specific, technically meaningful, and achievable objectives:

1. **Formal Capability Taxonomy and Intent Decomposition:** Establish an eight-class capability taxonomy covering filesystem, network, process execution, secrets, databases, dynamic runtimes, cloud APIs, and system metadata. Construct an automated LLM Intent Decomposer with constrained Pydantic schema validation to extract formal Expected Capability Specifications (C_{exp}) from natural-language prompts, handling ambiguous specifications via explicit UNKNOWN states.
2. **Multi-Language Static Capability Extraction:** Develop a multi-language static analysis engine based on Tree-sitter Concrete Syntax Tree (CST) queries to extract candidate execution authorities (C_{stat}) across JavaScript/TypeScript and Python codebases with precise source file and line-number attribution.
3. **Sandboxed Dynamic Observability Harness:** Construct an isolated, ephemeral Docker execution environment running rootless with dropped Linux capabilities (`--cap-drop=ALL`), read-only file system mounts, system-call tracing (`strace`), and outbound network proxy monitoring to capture concrete runtime behaviors (C_{dyn}) under least-privilege host constraints.
4. **Deterministic Alignment and Evidence Engine:** Design an alignment engine that computes capability set divergence (Δ), classifies operations into structured risk tiers (Justified, Conditionally Justified, Suspicious, Unjustified, Ambiguous), and maps findings directly to source code lines, system-call identifiers, and network endpoint destinations.
5. **Controlled Benchmark and Empirical Evaluation:** Curate a controlled benchmark suite of 50 multi-domain software applications across multiple AI code synthesis models (GPT-4o, Claude 3.5, DeepSeek-Coder) to evaluate the framework's precision, recall, F1-score, and false-positive rate against static-only baselines.

5. METHODOLOGY / PLANNING OF WORK

The project implements an empirical and experimental engineering methodology, structured across seven core research dimensions:

- **(a) Study Design:** A multi-stage verification framework that couples natural-language intent parsing with deterministic program analysis and dynamic runtime interposition.
- **(b) Study Settings:** Execution occurs within an ephemeral Docker container running rootless on a Linux host with a read-only root filesystem, dropped capabilities (`--cap-drop=ALL`), strict CPU/memory cgroups, and a 15-second execution timeout to prevent host leakage.
- **(c) Sampling and Benchmark Design:** A curated benchmark of 50 test applications across 7 operational categories: Clean Baselines (B1), Benign Extra Authority (B2), Hidden Malicious Capabilities (B3), Conditional Authority (B4), Dynamic Target Construction (B5), Multi-Model Variations (B6), and Adversarial Ambiguity (B7). Code is synthesized using leading LLM generators (GPT-4o, Claude 3.5, and DeepSeek-Coder).
- **(d) Variables:** *Independent:* Requirement specificity, AI model architecture, code complexity, injected capability divergence type. *Dependent:* Classification precision, recall, F1-score, false-positive rate, and execution latency.
- **(e) Controls and Baselines:** The system is evaluated against static-only SAST heuristics (AST pattern matching without intent context) and direct LLM-only review baselines.
- **(f) Study Methods and Architecture Pipeline:** The system processes inputs through a 5-stage pipeline:
 1. *Intent Decomposer:* Parses requirements into JSON schemas using Pydantic validation. Ambiguous prompts yield an UNKNOWN / INTENT_AMBIGUOUS state requesting clarification rather than generating false certainty.
 2. *Static Source Analyzer:* Tree-sitter CST queries locate capability-bearing AST nodes (e.g., `child_process.exec`, `fetch()`, `process.env`).
 3. *Dynamic Sandbox Tracing:* Linux `strace` captures system calls (`execve`, `connect`, `openat`), while an intercepting proxy logs DNS queries and HTTP endpoints.
 4. *Capability Normalizer & Alignment Engine:* Maps raw kernel traces and AST nodes into the unified 8-class taxonomy and evaluates set divergence (Δ).
 5. *Audit & Evidence Dashboard:* Visualizes capability graphs via Cytoscape/React Flow and generates automated CI/CD pull-request blocking reports.

5. Methodology / Planning of Work (Contd.)

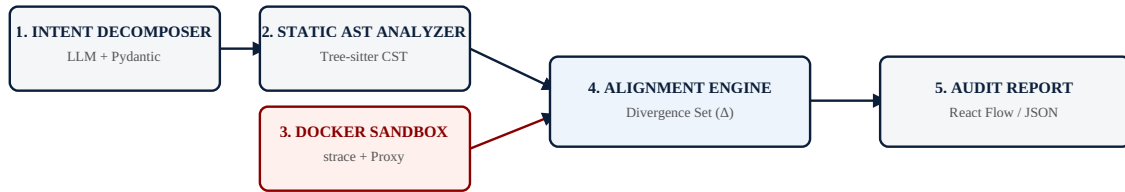


Figure 3: Intent2Capability System Architecture and Data Processing Pipeline

5.1 Semester-Wise Milestone Breakdown

- **7th Semester (Design, Extraction & Sandbox Prototype):** Formulation of the 8-class taxonomy; development of the LLM Intent Decomposer with Pydantic; implementation of Tree-sitter static extractors for JS/TS; construction of the Docker sandboxing harness with system-call tracing; Review-1 and Review-2 deliverables.
- **8th Semester (Alignment Engine, Full Benchmark & Defense):** Integration of the Graph Alignment Engine; PostgreSQL persistence tier; React Flow visualizer; execution of the 50-application benchmark suite; empirical comparative evaluation against baselines; final thesis documentation and Review-3/Final Viva defense.

6. ACTIVITY CHART: DETAILED WORK PLAN

Table 3: 16-Week Activity Chart and Milestone Plan

Timeline	Core Engineering Deliverables	Evaluation Milestones	Responsibility
Weeks 1–3	Freeze problem statement, threat model, 8-class taxonomy schema, and curate initial benchmark applications.	Review-0: Taxonomy signoff & benchmark specification freeze.	Team Lead & Module Leads
Weeks 4–7	Implement LLM Intent Decomposer with Pydantic validation; build Tree-sitter static extractors for JS/TS and Python.	Extract static capabilities from benchmark apps with >90% recall.	Intent & Static Analysis Leads
Weeks 8–10	Construct Docker sandbox execution harness; integrate Linux strace and proxy interception; set up PostgreSQL data tier.	Review-1 (Midterm): End-to-end trace validation on sample test suite.	Sandbox & Backend Leads
Weeks 11–13	Develop Graph Alignment Engine; build Cytoscape/React Flow UI dashboard; execute full 50-application benchmark.	Comparative evaluation of Precision, Recall, and F1 against baselines.	Alignment & UI Leads
Weeks 14–16	Address edge cases (conditional authority, obfuscation); offline Ollama fallback mode; documentation and viva rehearsal.	Final Defense: Complete demonstration and dissertation signoff.	All Team Members

REFERENCES

All references are listed in chronological order in accordance with the official GCET Vancouver format:

1. J.H. Saltzer and M.D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.
2. N. Provos, "Improving host security with system call policies," in *Proceedings of the 12th USENIX Security Symposium*, Washington, D.C., USA, Aug. 2003, pp. 257–272.
3. F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *Proceedings of the 2014 IEEE Symposium on Security and Privacy (S&P)*, San Jose, CA, USA, May 2014, pp. 590–604.
4. M. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proceedings of the 2022 IEEE Symposium on Security and Privacy (S&P)*, San Francisco, CA, USA, May 2022, pp. 754–768.
5. G. Sandoval, H. Pearce, T. Nisi, H. Kanal, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at C: A user study on the security implications of large language model code assistants," in *Proceedings of the 32nd USENIX Security Symposium*, Anaheim, CA, USA, Aug. 2023, pp. 2205–2222.
6. M.J.B. Kabore, R. Ramos, F. Al-Haidari, and S. Al-Dhubhani, "Security of AI-based source code generators: A systematic literature review," *ACM Computing Surveys*, vol. 56, no. 8, pp. 1–34, Feb. 2024.
7. T. Siddiq, S. Santos, V.S. Sharma, and C. Kaluza, "Empirical evaluation of code generation capabilities and privilege boundaries in Large Language Models," in *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, Lisbon, Portugal, Apr. 2024, pp. 112–124.
8. M.R. Chen, D. Wang, and Y. Liu, "Specification mining and behavioral intent verification for neural code synthesis," *IEEE Transactions on Software Engineering*, vol. 51, no. 3, pp. 610–625, Mar. 2025.